

Tournament Sort

锦标赛排序

本页面将简要介绍锦标赛排序。

定义

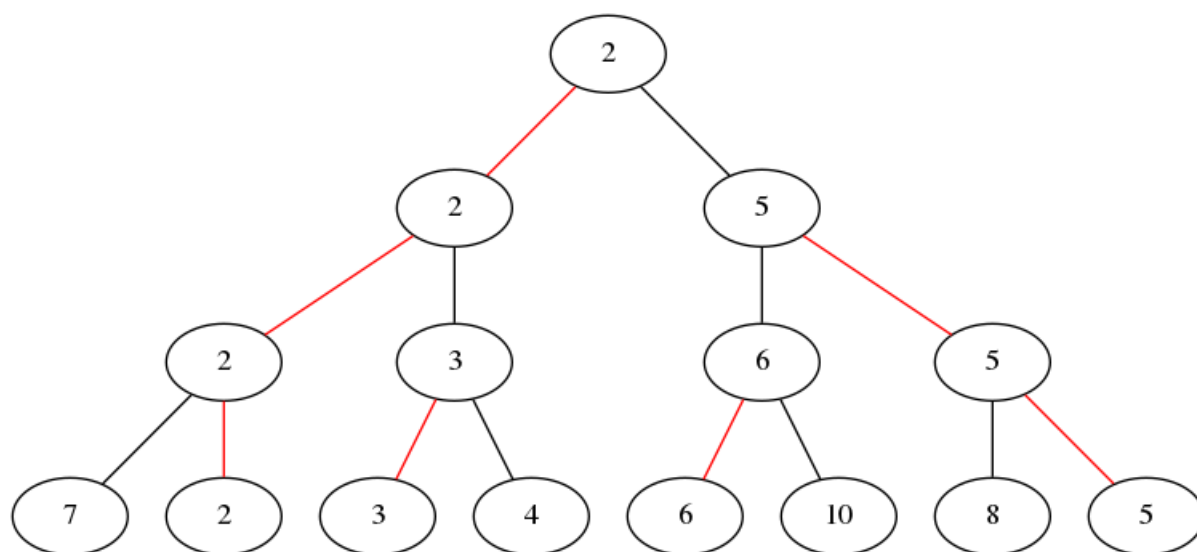
锦标赛排序（英文：Tournament sort），又被称为树形选择排序，是[选择排序](#)的优化版本，[堆排序](#)的一种变体（均采用完全二叉树）。它在选择排序的基础上使用优先队列查找下一个该选择的元素。

引入

锦标赛排序的名字来源于单败淘汰制的竞赛形式。在这种赛制中有许多选手参加比赛，他们两两比较，胜者进入下一轮比赛。这种淘汰方式能够决定最好的选手，但是在最后一轮比赛中被淘汰的选手不一定是第二好的——他可能不如先前被淘汰的选手。

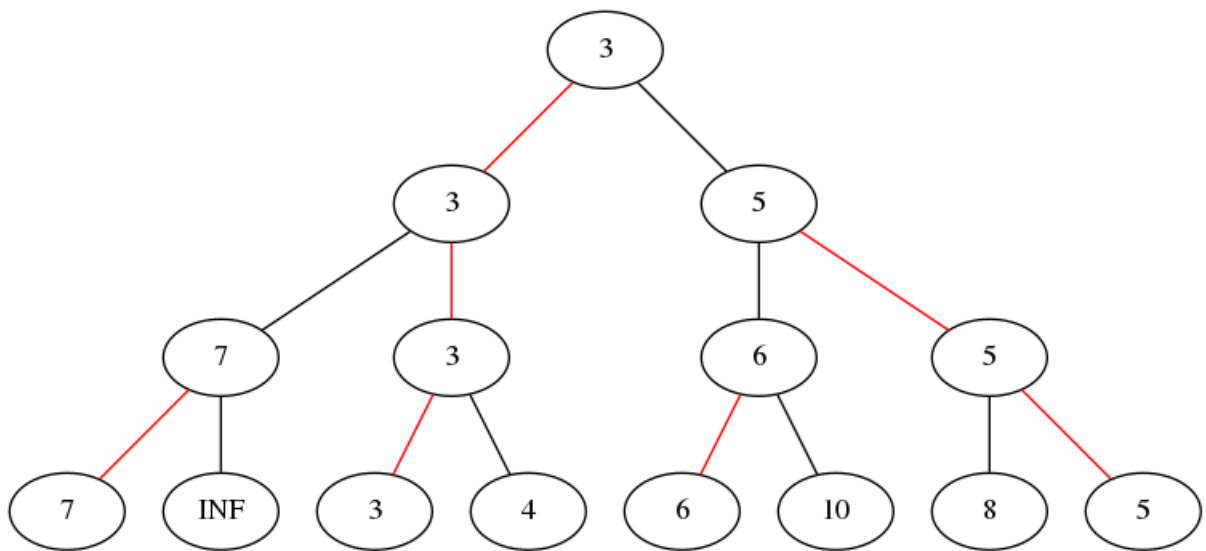
过程

以 **最小锦标赛排序树** 为例：



待排序元素是叶子节点显示的元素。红色边显示的是每一轮比较中较小的元素的胜出路径。显然，完成一次“锦标赛”可以选出一组元素中最小的那一个。

每一轮对 n 个元素进行比较后可以得到 $n/2$ 个「优胜者」，每一对中较小的元素进入下一轮比较。如果无法凑齐一对元素，那么这个元素直接进入下一轮的比较。



完成一次「锦标赛」后需要将被选出的元素去除。直接将其设置为 `INF`（这个操作类似 [堆排序](#)），然后再次举行「锦标赛」选出次小的元素。

之后一直重复这个操作，直至所有元素有序。

性质

稳定性

锦标赛排序是一种不稳定的排序算法。

时间复杂度

锦标赛排序的最优时间复杂度、平均时间复杂度和最坏时间复杂度均为 $O(n \log n)$ 。它用 $O(n)$ 的时间初始化「锦标赛」，然后用 $O(n \log n)$ 的时间从 n 个元素中选取一个元素。

空间复杂度

锦标赛排序的空间复杂度为 $O(n)$ 。

实现

🔗
C++Python

```

1 int n, a[MAXN], tmp[MAXN << 1];
2
3 int winner(int pos1, int pos2) {
4     int u = pos1 >= n ? pos1 : tmp[pos1];
5     int v = pos2 >= n ? pos2 : tmp[pos2];
6     if (tmp[u] <= tmp[v]) return u;
7     return v;
8 }
9
10 void creat_tree(int &value) {
11     for (int i = 0; i < n; i++) tmp[n + i] = a[i];
12     for (int i = 2 * n - 1; i > 1; i -= 2) {
13         int k = i / 2;
14         int j = i - 1;
15         tmp[k] = winner(i, j);
16     }
17     value = tmp[tmp[1]];
18     tmp[tmp[1]] = INF;
19 }
20
21 void recreat(int &value) {
22     int i = tmp[1];
23     while (i > 1) {
24         int j, k = i / 2;
25         if (i % 2 == 0)
26             j = i + 1;
27         else
28             j = i - 1;
29         tmp[k] = winner(i, j);
30         i = k;
31     }
32     value = tmp[tmp[1]];
33     tmp[tmp[1]] = INF;
34 }
35
36 void tournament_sort() {
37     int value;
38     creat_tree(value);
39     for (int i = 0; i < n; i++) {
40         a[i] = value;
41         recreat(value);
42     }
43 }

```

```

1 n = 0
2 a = [0] * MAXN
3 tmp = [0] * MAXN * 2
4
5
6 def winner(pos1, pos2):
7     u = pos1 if pos1 >= n else tmp[pos1]
8     v = pos2 if pos2 >= n else tmp[pos2]
9     if tmp[u] <= tmp[v]:
10         return u
11     return v
12
13
14 def creat_tree():
15     for i in range(0, n):
16         tmp[n + i] = a[i]
17     for i in range(2 * n - 1, 1, -2):
18         k = int(i / 2)
19         j = i - 1
20         tmp[k] = winner(i, j)
21     value = tmp[tmp[1]]
22     tmp[tmp[1]] = INF
23     return value
24
25
26 def recreat():
27     i = tmp[1]
28     while i > 1:
29         j = k = int(i / 2)
30         if i % 2 == 0:
31             j = i + 1
32         else:
33             j = i - 1
34         tmp[k] = winner(i, j)
35         i = k
36     value = tmp[tmp[1]]
37     tmp[tmp[1]] = INF
38     return value
39
40
41 def tournament_sort():
42     value = creat_tree()
43     for i in range(0, n):
44         a[i] = value
45         value = recreat()

```

外部链接

- [Tournament sort - Wikipedia](#)

本页面最近更新：2024/11/9 22:30:23, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [Alisahhh](#), [Chihsiao](#), [Enter-tainer](#), [iamtwz](#), [lr1d](#), [ksyx](#), [mcendu](#), [Menci](#), [ShaoChenHeng](#), [shawlleyw](#), [shenshuaijie](#), [Tiphereth-A](#), [Xeonacid](#), [zhoudavinci](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用

